```
    lp = len(pop)
    while len(new_pop) < len(pop):                          1
        rids = np.random.randint(low=0,high=lp, \
        size=(int(tournament_size*lp)))                     2
        batch = np.array([[i,x['fitness']] for \
        (i,x) in enumerate(pop) if i in rids])              3
        scores = batch[batch[:, 1].argsort()]               4
        i0, i1 = int(scores[-1][0]),int(scores[-2][0])      5
        parent0,parent1 = pop[i0],pop[i1]
        offspring_ = recombine(parent0,parent1)             6
        child1 = mutate(offspring_[0], rate=mut_rate)       7
        child2 = mutate(offspring_[1], rate=mut_rate)
        offspring = [child1, child2]
        new_pop.extend(offspring)
    return new_pop
```

- ***1*** While the new population is not full
- ***2*** Selects a percentage of the full population as a subset
- ***3*** Subsets the population to get a batch of agents and matches each one with their index value in the original population
- ***4*** Sorts this batch in increasing order of score
- ***5*** The last agents in the sorted batch are the agents with the highest scores; selects the top 2 as parents.
- ***6*** Recombines the parents to get offspring
- ***7*** Mutates the children before putting them into the next generation

The `next_generation` function creates a list of random indices to index the population list and create a subset for a tournament batch. We use the `enumerate` function to keep track of the index positions of each agent in the subset so we can refer back to them in the main population. Then we sort the batch of fitness scores in ascending order and take the last two elements in the list as the top two individuals in that batch. We look up their indices and select the whole agent from the original population list.

Putting it all together, we can train a population of agents to play CartPole in just a handful of generations. You should experiment with the hyperparameters of mutation rate, population size, and number of generations.

Listing 6.13. Training the models

```
num_generations = 25
```
1